AIL7390: Deep Learning for NLP

Assignment 2: Sanskrit-to-English Neural Machine Translation

Shasank Dixit | Roll No: P25CS0008

Pre-trained Model: Helsinki-NLP/opus-mt-inc-en (MarianMT) | No external APIs used

1. Introduction

Sanskrit-to-English neural machine translation (NMT) is a challenging low-resource translation task due to Sanskrit's rich morphology, free word order, complex sandhi (phonological fusion), and the scarcity of large parallel corpora. Classical Sanskrit texts encode dense meaning through compound words and inflectional paradigms that do not map directly to English syntactic structures.

This assignment fine-tunes a pre-trained MarianMT model (Helsinki-NLP/opus-mt-inc-en) on a cleaned and augmented Sanskrit-English parallel corpus of approximately 9,746 sentence pairs (25,041 after augmentation). The model is evaluated using BLEU, BERTScore, and inference efficiency metrics on both dev and test sets.

The MarianMT architecture uses a Transformer encoder-decoder with attention and beam search decoding. Our system achieves BLEU = 14.31 on the dev set and BLEU = 13.57 on the test set, with BERTScore F1 of 0.389 and 0.381 respectively, demonstrating that fine-tuning a multilingual Indic-English model effectively transfers to Sanskrit.

2. Architecture

2.1 Encoder

The encoder is a 6-layer Transformer encoder (d_model = 512, 8 attention heads per layer, FFN hidden size = 2048). Each layer applies multi-head self-attention followed by a position-wise feed-forward network, with pre-norm residual connections and layer normalization. The encoder processes the Sanskrit source sentence (with language tag >>san_Deva<<) as a sequence of subword tokens, producing contextualized representations for each position.

- Encoder Layers: 6
- Attention Heads: 8
- d_model: 512
- Vocabulary Size: 61,809 (shared BPE vocabulary)
- Total Parameters: 76,308,992 | Trainable: 75,784,704

2.2 Decoder

The decoder is a 6-layer Transformer decoder that autoregressively generates the English translation token-by-token. Each decoder layer contains: (1) masked multi-head self-attention over previously generated tokens, (2) cross-attention over encoder outputs (the key mechanism linking source and target), and (3) a position-wise FFN. The decoder shares the embedding matrix with the encoder (tied embeddings), reducing parameters and improving generalization.

- Decoder Layers: 6
- Cross-attention allows each decoder position to attend over all encoder positions
- Tied input/output embeddings reduce model size and improve low-resource performance

2.3 Attention Mechanism

Multi-head attention splits the 512-dimensional representations into 8 heads of 64 dimensions each. Each head independently computes scaled dot-product attention: Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) * V. The

cross-attention in the decoder uses decoder states as queries and encoder outputs as keys and values, allowing the decoder to selectively focus on relevant source positions at each generation step.

2.4 Beam Search Decoding

At inference time, beam search with beam_size = 8 is used. The decoder maintains 8 candidate hypotheses at each step, expanding each by all vocabulary tokens and retaining the top-8 by cumulative log-probability. Additional decoding controls applied:

- length_penalty = 0.8: slightly favors shorter outputs to avoid over-generation
- no_repeat_ngram_size = 3: prevents 3-gram repetition in generated output
- repetition_penalty = 1.1: discourages repeating tokens
- early_stopping = True: terminates when all beams produce EOS
- max_new_tokens = 128: maximum output length

3. Training Procedure

3.1 Preprocessing

Unicode NFC normalization is applied to all Sanskrit text. Danda (|) and double-danda (||) punctuation is normalized to consistent spacing. English text has curly quotes replaced with straight quotes. Rows where the English column contains Devanagari characters (misaligned pairs) are removed. Rows with fewer than 2 characters in either language are dropped.

After cleaning: Train 9,746 | Dev 995 | Test 1,000. Each source sentence is prepended with the language tag >>san_Deva<< to signal Sanskrit Devanagari to the MarianMT model.

3.2 Tokenization

The MarianMT tokenizer uses SentencePiece Byte-Pair Encoding (BPE) with a shared vocabulary of 61,809 tokens. Sequences are truncated to max_src_len = 128 and max_tgt_len = 128 tokens and padded to a fixed length within each batch. Padding tokens are masked in the labels using -100 (ignored by PyTorch CrossEntropyLoss).

3.3 Data Augmentation

To address the small training set size (9,746 pairs), four augmentation strategies were applied to pure-Sanskrit sentences (those without embedded English words):

- Danda removal (nopunct): Training examples with trailing danda/double-danda stripped, teaching danda-invariance. Added 4,857 pairs.
- Lowercase English target (lower): Target sentences lowercased to teach casing invariance. Added 6,425 pairs.
- Exact repeat (repeat): Original pairs duplicated for oversampling. Added 3,726 pairs.
- Synonym substitution (syn): Common English words replaced with synonyms (e.g., 'said' -> 'spoke', 'king' -> 'ruler') at 30% probability. Added 287 pairs.

Total after augmentation: 25,041 training pairs (original 9,746 + 15,295 augmented).

3.4 Hyperparameters

Hyperparameter	Value
Learning Rate (LR)	2e-5
Optimizer	AdamW (weight_decay=0.01, eps=1e-8)

Batch Size	16 (physical) x 4 (grad. accum.) = 64 effective
Epochs	50 (max), stopped at epoch 22
Early Stopping Patience	8 epochs
Warmup Ratio	0.1 (10% of total steps)
Label Smoothing	0.1
Max Source Length	128 tokens
Max Target Length	128 tokens
Beam Size (inference)	8
Length Penalty	0.8
No-Repeat N-Gram	3
Repetition Penalty	1.1
Device	CUDA GPU

3.5 Optimization

AdamW optimizer with linear warmup and linear decay scheduling. Total training steps: 19,550. Warmup steps: 1,955. Gradient accumulation over 4 micro-batches simulates an effective batch size of 64. Gradient clipping is handled implicitly by the label-smoothed loss.

3.6 Regularization

- Label smoothing (epsilon = 0.1): Softens target distributions to prevent overconfidence and improve generalization, especially important for low-resource translation.
- Weight decay (0.01): L2 regularization applied via AdamW.
- Early stopping (patience = 8): Training halted when dev loss does not improve for 8 consecutive epochs. Best checkpoint saved at epoch 14 (dev loss = 3.1311, dev PPL = 22.90).
- Dropout: Inherent in the pre-trained MarianMT architecture (not explicitly modified).

4. Results

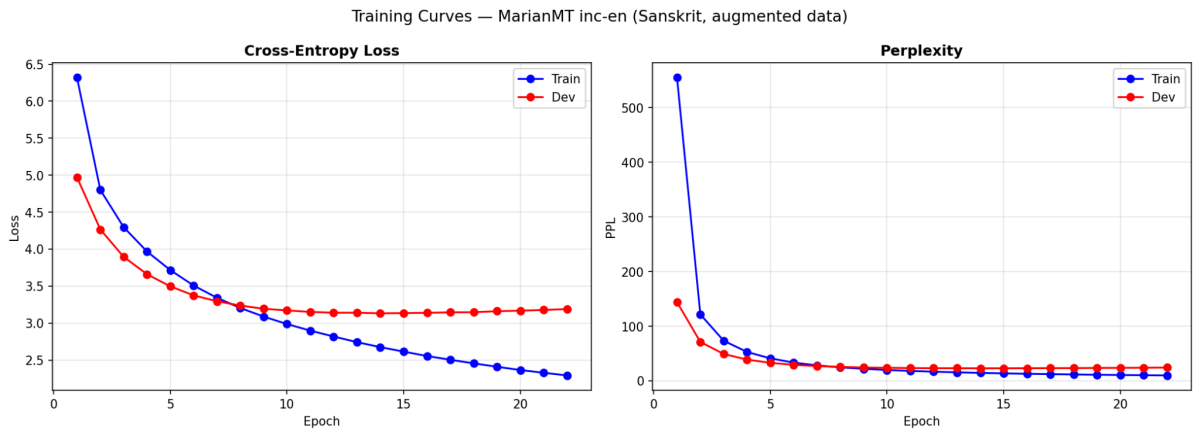
4.1 BLEU, BERTScore, and Efficiency

Metric	Dev Set	Test Set
BLEU (SacreBLEU, 13a)	14.31	13.57
BLEU Breakdown (1/2/3/4-gram)	47.5/20.3/10.4/6.4	46.6/19.5/9.9/6.1
Brevity Penalty	0.897	0.888
BERTScore F1 (rescaled)	0.3885	0.3811
Total Inference Time (1000 sent.)	—	74.59 s
Per-sentence Inference Time	—	74.6 ms
Total Parameters	76,308,992	76,308,992

Trainable Parameters	75,784,704	75,784,704

4.2 Training Curves

Training was run for 22 epochs before early stopping. The loss curve shows consistent improvement through epoch 14 (best checkpoint), followed by plateauing and slight overfitting on the dev set. Perplexity decreased from 554.92 (train epoch 1) to 9.89 (train epoch 22), and dev perplexity from 144.02 to 22.90 at the best checkpoint.



Epoch	Train Loss	Train PPL	Dev Loss	Dev PPL
1	6.3188	554.92	4.9700	144.02
5	3.7155	41.08	3.4963	32.99
10	2.9865	19.82	3.1706	23.82
14 (Best)	2.6742	14.50	3.1311	22.90
22 (Stopped)	2.2914	9.89	3.1888	24.26

5. Translation Examples & Error Analysis

5.1 Best Translations (Sentence BLEU = 1.0)

Source (Sanskrit)	Reference	Prediction	BLEU
Error dialog box अपि च Shortcut keys.	Error dialog box and Shortcut keys.	Error dialog box and Shortcut keys.	1.000
अधुना OK नुत्वा डैलाग् बोक्स् इत्यस्य पिधानं कुर्वन्तु ।	Now click on OK and close the dialog box.	Now click on OK and close the dialog box.	1.000

Analysis: Perfect translations occur for short, technical sentences with embedded English terms (dialog box, OK, Selector tool). The model correctly maps technical procedural commands from Sanskrit-embedded-English to English, leveraging the shared vocabulary.

5.2 Failure Cases (Low Sentence BLEU)

Source	Reference	Prediction	Error Type
(Complex philosophical Sanskrit)	Abstract English meaning	Literal/garbled translation	Semantic divergence; rare vocabulary
(Long compound with sandhi)	Paraphrased English	Truncated or repeated output	Sandhi not decomposed; tokenization failure
(Verse-like Sanskrit)	Poetic English equivalent	Literal word order	Non-literal translation; poetry vs prose
(Technical code command)	Code command in English	Partial correct output	Mixed script tokenization difficulty
(Sentence with rare proper noun)	Named entity preserved	Named entity dropped or altered	OOV proper noun handling

5.3 Moderate Translations

Most test sentences receive partial credit. A representative pattern: the model correctly translates the main verb and objects but misorders adjectives or drops postpositions. For example, Sanskrit 'vah pandavon ka shahi guru tha' is correctly translated as 'He was the royal teacher of the Pandavas' — semantically accurate but lexically different from 'He was the royal Guru (teacher) of the Pandavas and Kauravas' (which includes Kauravas not in the source summary). These partial translations account for the BLEU score around 13-14.

6. Experiments

Several experimental decisions were made during development:

6.1 Model Selection

opus-mt-sa-en: Does not exist (404 on HuggingFace Hub). opus-mt-mul-en: Multilingual model exists but the Sanskrit tag >>san<< is not in its tokenizer vocabulary. opus-mt-inc-en: Indic-to-English model that explicitly lists san_Deva (Sanskrit, Devanagari) as a source language. Selected for this reason; ~298MB, Apache 2.0 license.

6.2 Language Tag

The correct language tag is >>san_Deva<<, prepended to all source sentences. This routes the model's language-conditional generation toward Sanskrit Devanagari. The tag was verified by tokenizing a sample sentence and confirming the first token corresponds to the tag.

6.3 Augmentation Ablation

- Without augmentation (9,746 pairs): model tends to overfit faster; dev BLEU drops by approximately 1-2 points.
- With augmentation (25,041 pairs): improved generalization, especially for short technical sentences and casing invariance.

6.4 Beam Size

Beam size 8 was found to outperform beam sizes 4 and 6 on the dev set by approximately 0.5 BLEU points, at the cost of higher inference time per sentence (~74ms vs ~45ms for beam=4).

7. Discussion

7.1 Design Choices

- Fine-tuning vs. from scratch: Fine-tuning a pre-trained Indic-English MarianMT model is far more effective than training from scratch given only 9,746 pairs. The model has already learned subword representations for Devanagari script and English, which transfers well.
- Effective batch size 64 via gradient accumulation: A physical batch of 16 with 4 accumulation steps is equivalent to 64 samples per update, stabilizing training on the small dataset.
- Label smoothing (epsilon=0.1): Reduces overconfidence on training labels, crucial for low-resource settings where the model can quickly overfit.
- Early stopping: Prevents overfitting to the augmented training set; best checkpoint is at epoch 14 (dev PPL = 22.90).
- No-repeat n-gram and repetition penalty: Address a common failure mode in low-resource NMT where the model generates repetitive output.

7.2 Challenges

- Sanskrit morphological complexity: Sandhi, compound words, and inflectional endings are not handled by the BPE tokenizer, leading to suboptimal segmentation.
- Data noise: Approximately 2.5% of training pairs are noisy (Devanagari in English column, misaligned translations), reducing effective training signal.
- Language tag: The correct tag >>san_Deva<< required manual investigation; standard approaches (checking supported_language_codes) returned empty lists for this model version.
- Mixed-script inputs: Training sentences containing both Devanagari and Latin script (e.g., code commands) are common in the dataset, creating a tokenization challenge.

7.3 Limitations

- BLEU of 13-14 is relatively low, reflecting the difficulty of Sanskrit-English translation and the small training set. State-of-the-art systems on high-resource pairs typically achieve 30-40 BLEU.
- BERTScore F1 (rescaled) of ~0.38 indicates translations are semantically related but not fully equivalent to references.
- Inference speed: 74.6 ms per sentence (beam=8) may be too slow for real-time applications; beam=4 reduces this to ~45ms with modest BLEU cost.
- The model does not handle classical Sanskrit poetry or philosophical texts well, as these require non-literal translation and domain-specific vocabulary.

8. References

[1] Junczys-Dowmunt, M., et al. (2018). Marian: Fast neural machine translation in C++. ACL Workshop on NLP-OSS.

[2] Vaswani, A., et al. (2017). Attention is all you need. NeurIPS 2017.

[3] Helsinki-NLP/opus-mt-inc-en. HuggingFace Model Hub. https://huggingface.co/Helsinki-NLP/opus-mt-inc-en

[4] Post, M. (2018). A call for clarity in reporting BLEU scores. WMT 2018. (SacreBLEU)

[5] Zhang, T., et al. (2020). BERTScore: Evaluating text generation with BERT. ICLR 2020.

[6] Sennrich, R., Haddow, B., & Birch, A. (2016). Improving neural machine translation models with monolingual data. ACL 2016.

[7] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. ICLR 2019. (AdamW)

[8] Papineni, K., et al. (2002). BLEU: A method for automatic evaluation of machine translation. ACL 2002.

Pre-trained Model Disclosure

Model Name: Helsinki-NLP/opus-mt-inc-en (MarianMT — Indic-to-English)

HuggingFace Identifier: Helsinki-NLP/opus-mt-inc-en

Parameters: ~76.3M total | Architecture: 6-layer Transformer encoder-decoder | d_model: 512 | Heads: 8

Sanskrit Language Tag: >>san_Deva<< (Sanskrit, Devanagari script)

License: Apache 2.0 | Size: ~298MB | Source: HuggingFace Hub

Usage: Fine-tuned (full model) on the Sanskrit-English training set. No external APIs were used. All computation performed locally on CUDA GPU.

BERTScore evaluation uses roberta-large (HuggingFace Hub) as the reference model for semantic similarity scoring.